

SmartSafe Master Unit

Complete Design and Code Explanation of the Servo Lock Module

Microprocessors End-Term Project

July 2026

Abstract

This report documents the servo-lock subsystem of the SmartSafe Master unit. The actuator is controlled from the ATmega32 OC1A output on PD5 by Timer1 hardware PWM. The report explains how a hobby servo interprets pulse width, why a 50 Hz frame is used, how Fast PWM mode 14 is selected, how the Timer1 prescaler and ICR1 value produce an exact 20 ms frame at an 8 MHz CPU clock, how OCR1A generates the 1.0 ms, 1.5 ms, and 2.0 ms control pulses, and how the angle-to-pulse conversion is derived. It also explains the independent Timer2 countdown that keeps the safe unlocked for approximately five seconds, the interaction with the main loop, startup and power-restoration behavior, sleep handling, Proteus settings, electrical limitations, and practical verification methods.

Contents

1	Role of the Servo in SmartSafe	3
2	Hardware Connection	3
3	How a Hobby Servo Interprets the Signal	3
4	Why Timer1 Is Used	4
5	Timer1 Register Configuration	4
5.1	PD5 data direction	4
5.2	Waveform-generation mode	4
5.3	Non-inverting OC1A output	5
5.4	Clock prescaler	5
6	Calculation of ICR1 and the 50 Hz Frame	5
7	Calculation of OCR1A Values	6
8	Lock and Unlock Functions	6
9	Five-Second Open Interval	7
10	Timer2 Calculation for the Open-Time Counter	7
11	Main-Loop Sequence	8
12	Startup, Sleep, and Power-Failure Behavior	8
12.1	Startup	8
12.2	Normal inactivity sleep	8
12.3	Confirmed power failure	8

13 Proteus Configuration and Test Procedure	8
14 Common Failure Modes	9
15 Register Summary	9
16 Conclusion	9

1 Role of the Servo in SmartSafe

The servo represents the mechanical locking mechanism of the safe. The Master keeps the servo at the locked position during startup and ordinary operation. After a correct four-digit password is accepted, the Master commands the servo to the unlocked position, starts a five-second software countdown, blocks additional keypad activity during that interval, and then returns the actuator to the locked position.

The design separates two timing tasks:

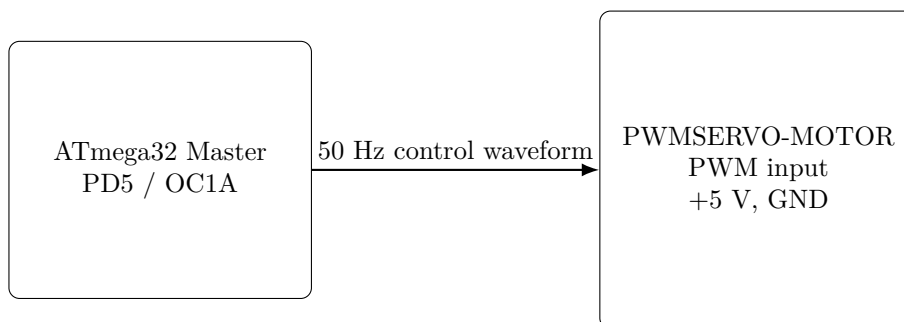
1. **Timer1** continuously generates the high-resolution PWM waveform required by the servo.
2. **Timer2** maintains the application-level five-second open interval.

Timer1 therefore determines the physical shaft position, while Timer2 determines how long the safe remains open.

2 Hardware Connection

The final signal connection is:

Servo connection	Master connection	Purpose
Control/PWM	PD5/OC1A	Timer1 compare output
Positive supply	+5 V	Servo electronics and motor supply
Ground	GND	Common electrical reference



Power supply and Master must share ground

Figure 1: Servo control connection.

In Proteus the model can usually be powered directly from the 5 V rail. In real hardware, a servo can draw a substantial startup or stall current. It should normally use a supply sized for the motor current, a local bulk capacitor, and a 100 nF decoupling capacitor. The servo ground and microcontroller ground must remain common. The servo must not be powered from an I/O pin.

3 How a Hobby Servo Interprets the Signal

A standard position servo does not interpret PWM duty cycle in the same way as a DC motor controller. It primarily measures the duration of each HIGH pulse. A new pulse is sent repeatedly within a frame of approximately 20 ms:

- approximately 1.0 ms HIGH: one end position, used as locked;

- approximately 1.5 ms HIGH: center position, used as unlocked in this project;
- approximately 2.0 ms HIGH: opposite end position.



Figure 2: Servo waveform concept. The illustrated pulse is schematic; the frame repeats continuously.

For the selected 20 ms frame, the nominal duty cycles are only 5 percent, 7.5 percent, and 10 percent. The pulse width, rather than the percentage itself, is the meaningful control quantity.

4 Why Timer1 Is Used

Timer1 is a 16-bit timer/counter. A 20 ms frame with one-microsecond resolution requires 20,000 timer counts, which is larger than the 256-count range of an 8-bit timer. Timer1 can represent this period directly and provides a dedicated hardware compare output on OC1A/PD5. Once configured, the waveform is generated in hardware without software toggling and without timing jitter from the main loop.

5 Timer1 Register Configuration

The initialization code is:

```

1 void servo_init(void)
2 {
3     SERVO_DDR |= (1U << SERVO_PIN);
4
5     TCCR1A = (1U << COM1A1) | (1U << WGM11);
6     TCCR1B = (1U << WGM13) |
7             (1U << WGM12) |
8             (1U << CS11);
9
10    ICR1 = 19999U;
11    servo_lock();
12 }
```

Listing 1: Servo initialization and Timer1 configuration.

5.1 PD5 data direction

```

1 SERVO_DDR |= (1U << SERVO_PIN);
```

sets DDRD5=1. This makes PD5 an output. When the compare-output function is enabled, Timer1 controls the pin automatically.

5.2 Waveform-generation mode

The waveform-generation bits are distributed across two registers:

Bit	WGM13	WGM12	WGM11	WGM10
Selected value	1	1	1	0

This binary pattern, 1110, selects Fast PWM mode 14. In this mode:

- ICR1 defines TOP and therefore the frame period;
- OCR1A defines the compare point and therefore the HIGH-pulse width;
- the counter runs from 0 through ICR1, then restarts at 0.

The register assignments are:

```
1 TCCR1A = (1U << WGM11);
2 TCCR1B = (1U << WGM13) | (1U << WGM12);
```

with the compare-output and prescaler bits added to the same values.

5.3 Non-inverting OC1A output

The compare-output bits are:

COM1A1	COM1A0	Behavior in Fast PWM
1	0	Clear OC1A on compare match; set OC1A at BOTTOM

Therefore the output becomes HIGH when the timer starts a new frame and becomes LOW when TCNT1=OCR1A. The HIGH duration is approximately OCR1A timer ticks.

5.4 Clock prescaler

The clock-select field is:

CS12	CS11	CS10	Timer1 clock
0	1	0	$F_{CPU}/8$

At $F_{CPU} = 8\,000\,000$ Hz,

$$f_{T1} = \frac{8\,000\,000}{8} = 1\,000\,000 \text{ Hz.}$$

The duration of one timer count is therefore

$$T_{tick} = \frac{1}{1\,000\,000} = 1 \mu s.$$

This prescaler is especially convenient because timer counts and pulse width in microseconds have the same numerical value.

6 Calculation of ICR1 and the 50 Hz Frame

In Fast PWM mode 14, the frame period is

$$T_{PWM} = \frac{N(1 + ICR1)}{F_{CPU}},$$

where N is the prescaler. Solving for ICR1 with a 20 ms target gives

$$ICR1 = \frac{T_{PWM}F_{CPU}}{N} - 1 = \frac{0.020 \times 8\,000\,000}{8} - 1 = 19\,999.$$

The resulting frequency is

$$f_{PWM} = \frac{8\,000\,000}{8(19\,999 + 1)} = 50 \text{ Hz.}$$

The code therefore uses:

```
1 ICR1 = 19999U;
```

7 Calculation of OCR1A Values

Because one Timer1 count equals one microsecond:

Commanded position	Pulse width	OCR1A	Duty cycle
Locked, 0 degrees	1.0 ms	1000	5.0 percent
Unlocked, 90 degrees	1.5 ms	1500	7.5 percent
Opposite end, 180 degrees	2.0 ms	2000	10.0 percent

The conversion function is:

```
1 void servo_set(uint8_t angle)
2 {
3     if (angle > 180U)
4         angle = 180U;
5
6     OCR1A = (uint16_t)(1000UL +
7                       ((uint32_t)angle * 1000UL) / 180UL);
8 }
```

Listing 2: Angle-to-pulse conversion.

The linear mapping is

$$OCR1A = 1000 + \frac{angle \times 1000}{180}.$$

Examples are:

$$\begin{aligned} angle = 0 : \quad OCR1A &= 1000, \\ angle = 90 : \quad OCR1A &= 1000 + \frac{90 \times 1000}{180} = 1500, \\ angle = 180 : \quad OCR1A &= 2000. \end{aligned}$$

The multiplication is explicitly promoted to 32 bits so that the intermediate product cannot overflow an 8-bit or 16-bit object.

8 Lock and Unlock Functions

The public interface deliberately hides numeric angles from the application:

```
1 void servo_lock(void)
2 {
3     servo_set(0U);
4 }
5
6 void servo_unlock(void)
7 {
8     servo_set(90U);
9 }
```

The project defines 0 degrees as locked and 90 degrees as unlocked. This convention can be reversed mechanically without changing the Timer1 calculations; only the two commanded angles would need adjustment.

9 Five-Second Open Interval

Timer1 controls position but does not count the five-second access interval. The main application uses:

```
1 servo_unlock();
2 servo_open = 1U;
3 servo_counter = SERVO_OPEN_SEC; /* 5 */
```

Timer2 decrements `servo_counter` approximately once per second. The main loop then detects zero and locks the servo:

```
1 if (servo_open && servo_counter == 0U) {
2     servo_lock();
3     servo_open = 0U;
4     redraw = 1U;
5 }
```

Keeping the actual actuator command in the main loop prevents complex peripheral operations from being performed inside an interrupt service routine.

10 Timer2 Calculation for the Open-Time Counter

Timer2 is configured in CTC mode:

```
1 OCR2 = 77U;
2 TCCR2 = (1U << WGM21) |
3         (1U << CS22) |
4         (1U << CS21) |
5         (1U << CS20);
6 TIMSK |= (1U << OCIE2);
```

The clock-select bits 111 select a prescaler of 1024. The compare period is

$$T_{match} = \frac{1024(77 + 1)}{8\,000\,000} = 9.984 \text{ ms.}$$

Ten compare matches produce

$$10 \times 9.984 = 99.84 \text{ ms,}$$

and ten 100 ms software intervals produce

$$10 \times 99.84 = 998.4 \text{ ms.}$$

Thus the nominal five-second interval is approximately

$$5 \times 0.9984 = 4.992 \text{ s.}$$

The error is only about -0.16 percent, which is insignificant for a safe-door demonstration.

11 Main-Loop Sequence

A successful password causes the following sequence:

1. reset the internal EEPROM failed-attempt count;
2. store a successful-login event in the external EEPROM;
3. send `LOGIN_OK` through USART to the Slave;
4. call `servo_unlock()`, loading `OCR1A=1500`;
5. set `servo_open=1` and `servo_counter=5`;
6. prevent keypad input and inactivity sleep while the door is open;
7. decrement the counter through Timer2;
8. call `servo_lock()` when the counter reaches zero.

12 Startup, Sleep, and Power-Failure Behavior

12.1 Startup

`servo_init()` configures Timer1 and immediately calls `servo_lock()`. The actuator therefore begins in a defined safe position rather than using an undefined compare value.

12.2 Normal inactivity sleep

Normal inactivity sleep is allowed only when `servo_open=0`. The safe therefore cannot enter Power-down while the door-open countdown is active.

12.3 Confirmed power failure

Before power-failure standby, the firmware stops Timer1 and forces the servo signal pin low. After monitored voltage restoration, `servo_init()` is called again, which reconstructs the 50 Hz waveform and locks the safe. This fail-safe restart behavior is intentional.

13 Proteus Configuration and Test Procedure

Use the following settings:

- ATmega32 clock property: 8 MHz;
- servo signal: PD5/OC1A;
- frame period: 20 ms or frequency 50 Hz;
- minimum pulse: approximately 1 ms;
- center pulse: approximately 1.5 ms;
- maximum pulse: approximately 2 ms;
- initial angle: locked position.

A logic analyzer should show a 20 ms period. In the locked state the HIGH pulse should be about 1 ms. After password acceptance it should become about 1.5 ms for approximately five seconds and then return to 1 ms.

14 Common Failure Modes

Symptom	Likely cause and correction
Servo remains near 180 degrees	MCU clock does not match <code>F_CPU</code> , Timer1 prescaler is wrong, or the Proteus pulse limits do not match 1–2 ms.
Movement lasts about 40 seconds	The MCU is operating near 1 MHz while the firmware calculations assume 8 MHz.
Servo jitters	Inadequate supply current, missing common ground, noisy supply, or software control rather than hardware OC1A.
No waveform on PD5	<code>DDRD5</code> is not output, compare output is disabled, or Timer1 is stopped.
Wrong mechanical direction	Reverse the lock/unlock angles or rotate the servo linkage; Timer1 calculations remain unchanged.
Servo resets after power restoration	This is intentional: reinitialization returns it to the locked state.

15 Register Summary

Register	Selected value/bits	Purpose
<code>DDRD</code>	<code>DDD5=1</code>	Make OC1A pin an output.
<code>TCCR1A</code>	<code>COM1A1=1, WGM11=1</code>	Non-inverting OC1A and part of mode 14.
<code>TCCR1B</code>	<code>WGM13=1, WGM12=1, CS11=1</code>	Complete mode 14 and select clock /8.
<code>ICR1</code>	19999	TOP for a 20 ms, 50 Hz frame.
<code>OCR1A</code>	1000–2000	Pulse width in microseconds.
<code>TCCR2</code>	<code>CTC, clock /1024</code>	Application timing for the five-second interval.
<code>OCR2</code>	77	Approximately 9.984 ms compare period.
<code>TIMSK</code>	<code>OCIE2=1</code>	Enable Timer2 compare interrupt.

16 Conclusion

The servo module uses Timer1 in Fast PWM mode 14 because a 16-bit hardware timer can generate a stable 20 ms frame with one-microsecond pulse resolution. The /8 prescaler converts the 8 MHz system clock into a 1 MHz Timer1 clock, `ICR1=19999` establishes 50 Hz, and `OCR1A` directly represents the required pulse width. Timer2 independently enforces the five-second access period, allowing the actuator waveform to remain continuous while the application performs other work. The result is deterministic, non-blocking, and appropriate for both Proteus simulation and a conventional hobby-servo interface.

References

- [1] Microchip Technology Inc., *ATmega32A Data Sheet*, sections on Timer/Counter1, Timer/Counter2, I/O ports, and sleep modes.
[Official Microchip PDF](#)

- [2] Microprocessors End-Term Project, *Smart Safe with Remote Alarm*, project specification supplied with the implementation.
- [3] SmartSafe source files: `servo.c`, `servo.h`, `timer.c`, `config.h`, and `main.c`, July 2026 project version.